

✓ EXERCISE 5: Regularization and Hyper parameter tuning

Goal:

Predict salary (continuous value) using regression models and analyze:

- underfitting vs overfitting
- effect of polynomial complexity
- L2 (Ridge) and L1 (Lasso) regularization
- model generalization on unseen data
- hyperparameter tuning using GridSearchCV

Dataset Features:

Years of Experience

Target: Salary

Step 0: Import Libraries: Import all the necessary libraries

Step 1: Load Dataset

Step 2: Train–Test Split

Step 3: Build & Evaluate Degree-10 Model (Overfitting Case)

Step 4: Visualize Degree-10 Model

Step 5: Apply Ridge Regression (L2 Regularization)

Step 6: Visualize Ridge Model

Step 7: Apply Lasso Regression (L1 Regularization)

Step 8: Visualize Lasso Model

Step 9: Hyperparameter Tuning using GridSearchCV

Step 10: Train Final Model Using Best GridSearch Parameter

✓ Step 1: Import Libraries: Import all the necessary libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```



```

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
from sklearn.model_selection import KFold

```

✓ Step 1: Load Dataset

Read the Salary_Data.csv file into a pandas DataFrame df. Extract YearsExperience into X and Salary into y for modeling.

```

# -----
# 1. Load Data
# -----
df = pd.read_csv("Salary_Data.csv")

X = df[["YearsExperience"]].values
y = df["Salary"].values

```

✓ Step 2: Train–Test Split

Split the dataset into training and testing sets using a 75%–25% ratio. Store the results in X_train, X_test, y_train, and y_test.

```

# -----
# 2. Train / Test Split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

```

```

# Create X_line as a sequence of evenly spaced values across the feature ran
X_line = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)

```

✓ Step 3: Build & Evaluate Degree-10 Model (Overfitting Case)

Unsupported Cell Type. Double-Click to inspect/edit the content.

```

# =====
# 5. OVERFITTING (Degree = 10)

```



```
# =====
poly10 = PolynomialFeatures(degree=10, include_bias=False)

Xtr10 = poly10.fit_transform(X_train)
Xte10 = poly10.transform(X_test)
Xline10 = poly10.transform(X_line)

lr10 = LinearRegression()
lr10.fit(Xtr10, y_train)

print("\nDegree 10 (Complex)")
print("Train R2:", r2_score(y_train, lr10.predict(Xtr10)))
print("Test R2:", r2_score(y_test, lr10.predict(Xte10)))
```

```
Degree 10 (Complex)
Train R2: 0.9865891156237337
Test R2: 0.8417511739182677
```

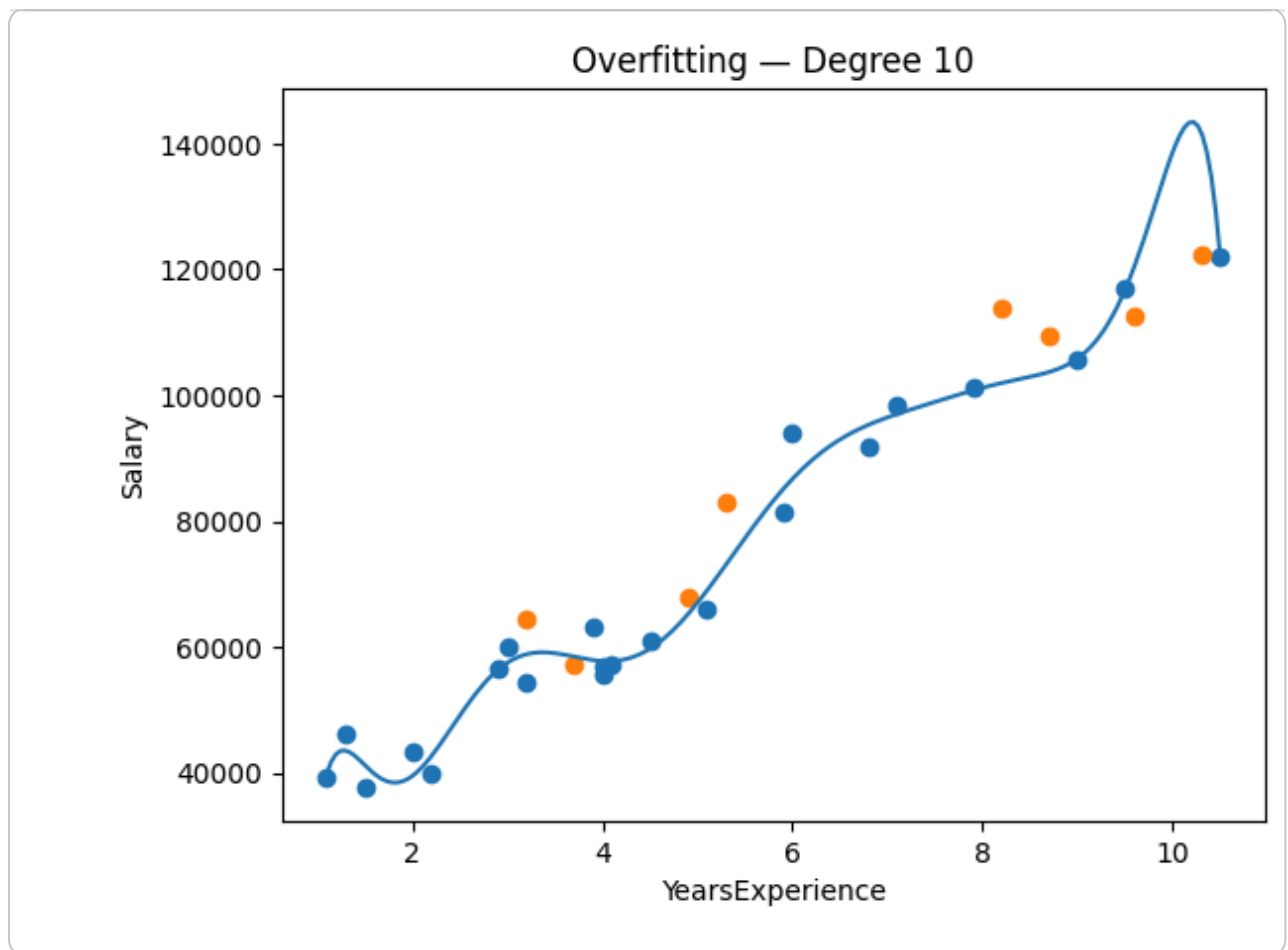
✓ Step 4: Visualize Degree-10 Model

Create a new figure for plotting.

- Plot the training and testing data points.
- Draw the fitted curve using `X_line` and predictions from `lr10` on `Xline10`.
- Add title and axis labels to the plot.
- Display the figure.

```
plt.figure()
plt.scatter(X_train, y_train)
plt.scatter(X_test, y_test)
plt.plot(X_line, lr10.predict(Xline10))
plt.title("Overfitting – Degree 10")
plt.xlabel("YearsExperience")
plt.ylabel("Salary")
plt.show()
```





✓ Step 5: Apply Ridge Regression (L2 Regularization)

Scale the polynomial features using StandardScaler, storing the results in `Xtr_scaled`, `Xte_scaled`, and `Xline_scaled`.

- Train a Ridge Regression model `ridge` with `alpha = 1` using `Xtr_scaled` and `y_train`.
- Predict outputs for training and testing data.
- Print Train and Test R^2 scores to evaluate the effect of L2 regularization on model performance.

```
# =====
# 6. RIDGE (L2)
# =====
scaler = StandardScaler()

Xtr_scaled = scaler.fit_transform(Xtr10)
Xte_scaled = scaler.transform(Xte10)
Xline_scaled = scaler.transform(Xline10)

ridge = Ridge(alpha=1)
ridge.fit(Xtr_scaled, y_train)

print("\nRidge (alpha = 1)")
print("Train R2:", r2_score(y_train, ridge.predict(Xtr_scaled)))
```



```
print("Test R2:", r2_score(y_test, ridge.predict(Xte_scaled)))
```

```
Ridge (alpha = 1)  
Train R2: 0.9588041229061915  
Test R2: 0.9324199379241933
```

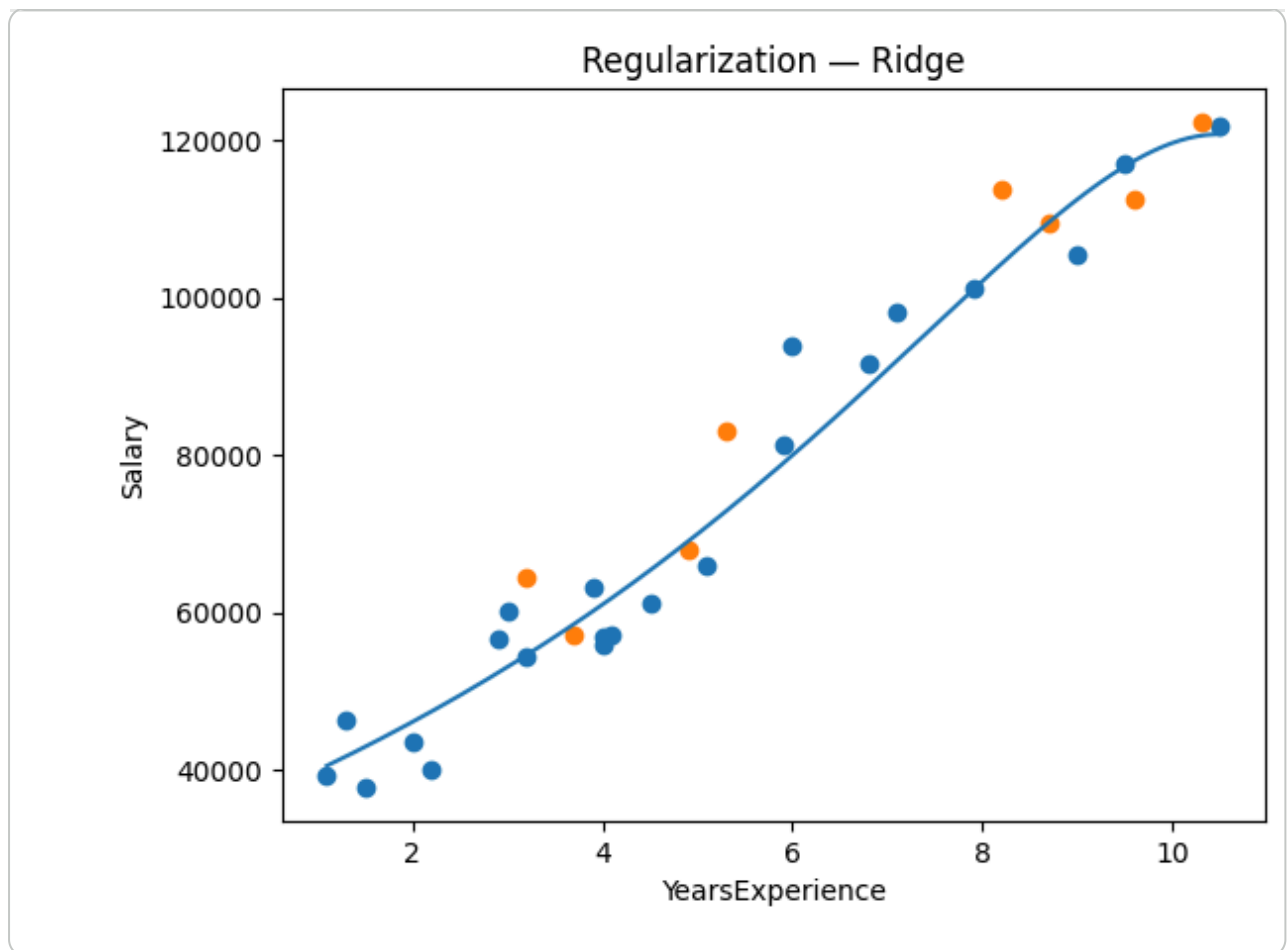
✓ Step 6: Visualize Ridge Model

Create a new figure for plotting.

- Plot the training data points (X_train, y_train).
- Plot the testing data points (X_test, y_test).
- Draw the regularized curve using X_line and predictions from ridge on Xline_scaled.
- Add title and axis labels to the plot.
- Display the figure.

```
plt.figure()  
plt.scatter(X_train, y_train)  
plt.scatter(X_test, y_test)  
plt.plot(X_line, ridge.predict(Xline_scaled))  
plt.title("Regularization – Ridge")  
plt.xlabel("YearsExperience")  
plt.ylabel("Salary")  
plt.show()
```





✓ Step 7: Apply Lasso Regression (L1 Regularization)

Train a Lasso Regression model lasso with $\alpha = 0.1$ using the scaled training features `Xtr_scaled` and `y_train`.

- Predict outputs for training and testing data.
- Print Train and Test R^2 scores to evaluate the effect of L1 regularization on model performance.

```
# =====
# 6. LASSO (L1)
# =====
lasso = Lasso(alpha=0.1, max_iter=10000)
lasso.fit(Xtr_scaled, y_train)

print("\nLasso (alpha = 0.1)")
print("Train R2:", round(r2_score(y_train, lasso.predict(Xtr_scaled)), 3))
print("Test R2:", round(r2_score(y_test, lasso.predict(Xte_scaled)), 3))
```

```
Lasso (alpha = 0.1)
Train R2: 0.966
Test R2: 0.939
```



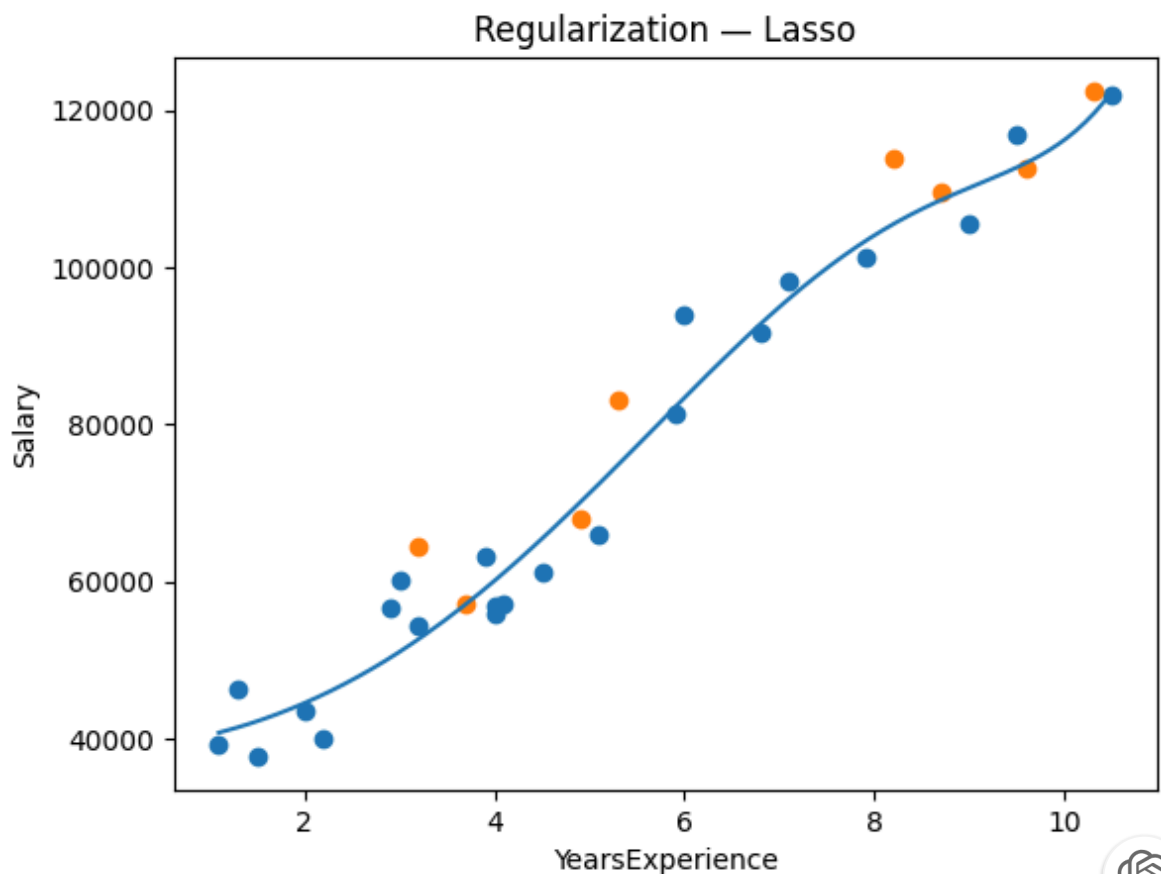
```
/opt/jupyterhub/lib/python3.10/site-packages/sklearn/linear_model/_coordinate  
model = cd_fast.enet_coordinate_descent(
```

✓ Step 8: Visualize Lasso Model

Create a new figure for plotting.

- Plot the training data points (X_{train} , y_{train}).
- Plot the testing data points (X_{test} , y_{test}).
- Draw the L1-regularized curve using X_{line} and predictions from lasso on $X_{\text{line_scaled}}$.
- Add title and axis labels to the plot.
- Display the figure.

```
plt.figure()  
plt.scatter(X_train, y_train)  
plt.scatter(X_test, y_test)  
plt.plot(X_line, lasso.predict(Xline_scaled))  
plt.title("Regularization – Lasso")  
plt.xlabel("YearsExperience")  
plt.ylabel("Salary")  
plt.show()
```



✓ Step 9: Hyperparameter Tuning using GridSearchCV

Hyperparameter tuning is the process of selecting the optimal values for a machine learning model's hyperparameters. These are typically set before the actual training process begins and control aspects of the learning process itself.

Effective tuning helps the model learn better patterns, avoid overfitting or underfitting and achieve higher accuracy on unseen data.

Techniques for Hyperparameter Tuning Models can have many hyperparameters and finding the best combination of parameters can be treated as a search problem. The two best strategies for Hyperparameter tuning are:

1. GridSearchCV

2. RandomSearchCV

✓ 1. GridSearchCV

GridSearchCV is a brute-force technique for hyperparameter tuning. It trains the model using all possible combinations of specified hyperparameter values to find the best-performing setup. It is slow and uses a lot of computer power which makes it hard to use with big datasets or many settings. It works using below steps:

Create a grid of potential values for each hyperparameter. Train the model for every combination in the grid. Evaluate each model using cross-validation. Select the combination that gives the highest score.



✓ 2. RandomizedSearchCV

As the name suggests RandomizedSearchCV picks random combinations of hyperparameters from the given ranges instead of checking every single combination like GridSearchCV.

In each iteration it tries a new random combination of hyperparameter values. It records the model's performance for each combination. After several attempts it selects the best-performing set.

Unsupported Cell Type. Double-Click to inspect/edit the content.

```
# =====  
# 7. GRID SEARCH (alpha tuning)
```




```
# =====
ridge2 = Ridge()

param_grid = {"alpha": [0.01, 0.1, 1, 10, 100]}

cv = KFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(ridge2, param_grid, cv=cv, scoring="r2")

grid.fit(Xtr_scaled, y_train)

print("\nGrid Search Result")
print("Best alpha:", grid.best_params_)
print("Best CV R2:", round(grid.best_score_, 3))
```

```
Grid Search Result
Best alpha: {'alpha': 1}
Best CV R2: 0.894
```

✓ Step 10: Train Final Model Using Best GridSearch Parameter

Extract the best alpha value from `grid.best_params_` and store it in `best_alpha`.

- Create a new Ridge model `final_ridge` using this optimal alpha.
- Train the final model on the scaled training data `Xtr_scaled` and `y_train`.
- Predict outputs for the plotting data `Xline_scaled` and store them in `yline_final`.
- Compute and print the final Train and Test R^2 scores using the tuned model.
- Plot the training and testing points along with the final fitted curve.
- Add title and axis labels to the plot.
- Display the figure.

```
# -----
# FINAL MODEL USING BEST ALPHA
# -----

best_alpha = grid.best_params_["alpha"]

print("\nRefitting Ridge using best alpha from Grid Search:", best_alpha)

final_ridge = Ridge(alpha=best_alpha)
final_ridge.fit(Xtr_scaled, y_train)

# Predict smooth curve
yline_final = final_ridge.predict(Xline_scaled)

print("Train R2:", round(r2_score(y_train, final_ridge.predict(Xtr_scaled)),
```



```
print("Test R2:", round(r2_score(y_test, final_ridge.predict(Xte_scaled)),
```

Refitting Ridge using best alpha from Grid Search: 1
Train R2: 0.959
Test R2: 0.932

T **B** *I* <> ⇌ 🖼️ “ 1/3 ⋮ — Ψ 😊 📄 [Close](#)

Plot the training and testing points along

- Add title and axis labels to the plot.
- Display the figure.

Plot the training and testing points along with the final fitted curve.

- Add title and axis labels to the plot.
- Display the figure.

```
# Plot final tuned model
plt.figure()
plt.scatter(X_train, y_train)
plt.scatter(X_test, y_test)
plt.plot(X_line, yline_final)
plt.title("Final Model After Grid Search (Best Ridge)")
plt.xlabel("YearsExperience")
plt.ylabel("Salary")
plt.show()
```

